Artificial Intelligence

Intelligent agents

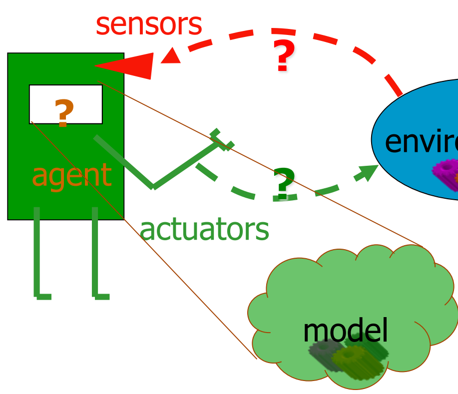
Dr. P. Victer Paul, Indian Institute of Information Technology Kottayam

contents

- Agents And Environments
- Agent Function and Program
- Rational agent
- Agent Program and types
- State Representation

Course – Introduction (recall)

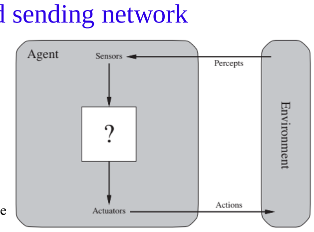
- The main unifying theme is the idea of an **intelligent agent**.
- define AI as the study of agents that receive percepts from the environment and perform actions.



Agents And Environments

- An agent is anything that can be viewed as perceiving its environment through sensors
- designed to automate tasks, make decisions, and interact with their environment to achieve specific goals.
- acting upon that environment through actuators.
  - A robotic agent might have cameras and infrared range finders for sensors and various motors for actuators.
  - A software agent receives keystrokes, file contents, and network packets as sensory inputs and acts on the environment by displaying on the screen, writing files, and sending network packets.

Intelligent agents are supposed to maximize their performance measure.



Functions and Examples

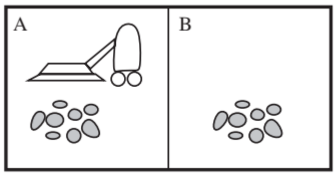
- Functions of Agents
  - Perception: Collect data from the environment using sensors.
  - Processing: Analyze the collected data to make decisions.
  - Action: Perform actions in the environment through actuators.
  - Learning: Improve performance over time by learning from experiences.
- Examples of Agents
  - Software Agents: Chatbots, virtual assistants (e.g., Siri, Alexa)
  - Robotic Agents: Autonomous robots, drones
  - Web Agents: Web crawlers, recommendation systems

Agent Function and Program

- percept to refer to the agent's perceptual inputs at any given instant
- percept sequence is the complete history of everything the agent has ever perceived.
- Thus, *an agent's choice of action at any given instant can depend on the entire percept sequence observed to date, but not on anything it hasn't perceived.*
- an agent's behavior is described by the **agent function** that maps any given percept sequence to an action
- the agent function for an artificial agent will be implemented by an **agent program**

The agent function is an abstract mathematical description;
the agent program is a concrete implementation, running
within some physical system.

Agent Function



A vacuum-cleaner world with just two locations

Percept sequence	Action
[*A*, *Clean*]	*Right*
[*A*, *Dirty*]	*Suck*
[*B*, *Clean*]	*Left*
[*B*, *Dirty*]	*Suck*
[*A*, *Clean*], [*A*, *Clean*]	*Right*
[*A*, *Clean*], [*A*, *Dirty*]	*Suck*
⋮	⋮
[*A*, *Clean*], [*A*, *Clean*], [*A*, *Clean*]	*Right*
[*A*, *Clean*], [*A*, *Clean*], [*A*, *Dirty*]	*Suck*
⋮	⋮

a simple agent function for the vacuum-cleaner world

Agent Program

A vacuum-cleaner world with just two locations

function REFLEX-VACUUM-AGENT([*location*,*status*]) **returns** an action

  if *status* = *Dirty* **then return** *Suck*
  else if *location* = *A* **then return** *Right*
  else if *location* = *B* **then return** *Left*

agent program for the two-state vacuum-cleaner world

Rational agent

one that does the right thing

- An agent generates a sequence of actions causes the environment to go through a sequence of states
  - **performance measure** that evaluates any given sequence of environment states
- *a general rule, it is better to design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave.*

Rational agent

- *Def:*

  For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

- *distinguish between rationality and omniscience*
- agent knows the *actual* outcome of its actions and can act accordingly
  - perfect knowledge about the environment, past, present, and future
  - All-knowing — unrealistic for practical AI

Nature Of Environments

- task environments are essentially the "problems" to which rational agents are the "solutions."
- Specifying the task environment
  - PEAS (Performance, Environment, Actuators, Sensors)

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Rational agent

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Properties of task environments

- *Observable or partially observable?*
- *Discrete or Continuous?*
- *Deterministic or Stochastic?*
- *Static or Dynamic?*
- *Episodic or Sequential?*
- *Multiple or Single Agent?*
- *Competitive or Cooperative Agent?*

Properties of task environments

Property	Definition	Examples
Observable vs Partially Observable	Fully observable: agent has complete access to environment state. Partially observable: agent sees only part of the environment.	Observable: Chess game. Partially observable: Poker, driving in fog.
Discrete vs Continuous	Discrete: finite, countable states and actions. Continuous: infinite possibilities.	Discrete: Tic-tac-toe, turn-based board games. Continuous: Self-driving car (speed, steering, position).
Deterministic vs Stochastic	Deterministic: outcome of actions is predictable. Stochastic: outcome involves randomness or uncertainty.	Deterministic: Solving a math equation. Stochastic: Weather forecasting, stock trading.
Static vs Dynamic	Static: environment doesn't change while agent is thinking. Dynamic: environment changes continuously.	Static: Crossword puzzle. Dynamic: Real-time traffic navigation.
Episodic vs Sequential	Episodic: each decision is independent. Sequential: past actions affect future outcomes.	Episodic: Image classification tasks. Sequential: Chess, job scheduling.
Single-agent vs Multi-agent	Single agent: only one decision-maker. Multi-agent: multiple agents interacting.	Single: Sudoku solver. Multi-agent: Online multiplayer games.
Competitive vs Cooperative (Multi-agent)	Competitive: agents compete against each other. Cooperative: agents work together toward a goal.	Competitive: Chess, sports. Cooperative: Multi-robot warehouse systems.

Properties of task environments

Attribute	Observable/ Partially Observable	Discrete/ Continuous	Deterministic/ Stochastic	Static/ Dynamic	Episodic/ Sequential	Single/ Multiple Agent	Competitive/ Cooperative
Example	Chess (Observable) / Poker (Partially Observable)	Tic-Tac-Toe (Discrete) / Autonomous Driving (Continuous)	Robotic Arm (Deterministic) / Weather Prediction (Stochastic)	Sudoku (Static) / Stock Trading (Dynamic)	Medical Diagnosis (Episodic) / Video Game (Sequential)	Email Filter (Single) / Multi-Player Game (Multiple)	Chess Match (Competitive) / Robot Assembly (

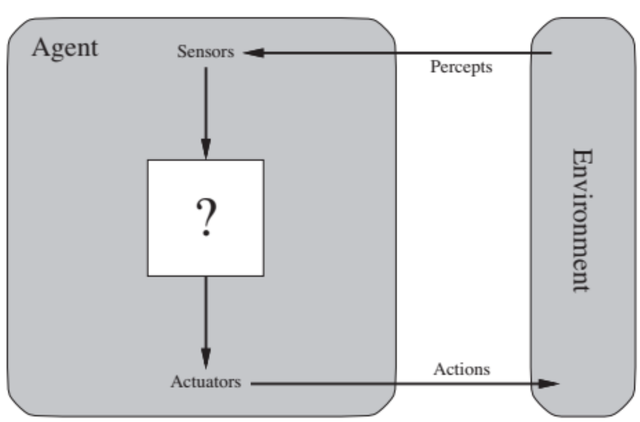
Properties of task environments

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Structure of Agents

- Sensors
  - Collect data from the environment.
  - Examples: Cameras, microphones, temperature sensors.
- Actuators
  - Perform actions based on decisions.
  - Examples: Motors, displays, speakers.
- Perception Module
  - Processes sensory data to understand the environment.
- Decision-Making Module
  - Uses algorithms and rules to make decisions.
  - Incorporates reasoning and planning capabilities.
- Learning Module
  - Improves agent performance through learning algorithms.
  - Adapts to new data and experiences.

Structure of Agents



Intelligent Agent Models

- The job of AI is to design an agent program that implements the agent function — the mapping from percepts to actions.
- program will run on some sort of computing device with physical sensors and actuators — architecture

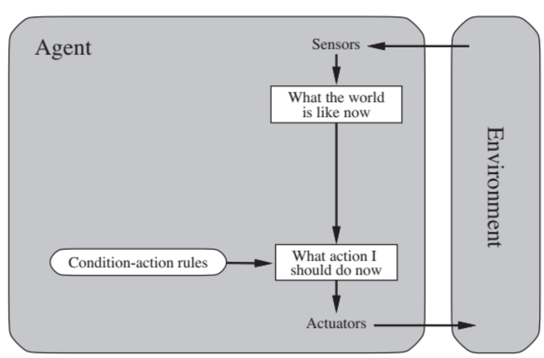
$$agent = architecture + program$$

- Five basic kinds of agent programs
  - Simple reflex agents;
  - Model-based reflex agents;
  - Goal-based agents; and
  - Utility-based agents
  - Learning based agents
- Each kind of agent program combines particular components in particular ways to generate actions

https://www.simform.com/blog/types-of-ai-agents/

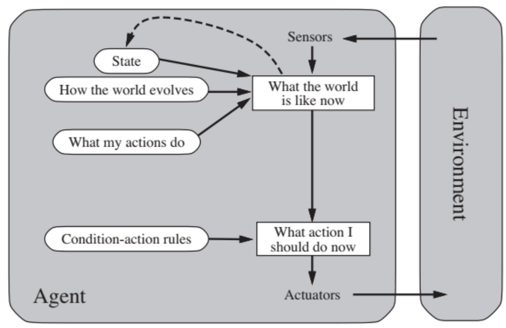
Simple reflex agents

- agents select actions based on the **current percept**, ignoring the rest of the percept history
- **condition–action rule**
  - *A thermostat that turns a heater on if the temperature drops below a set threshold.*



Model-based reflex agents

- knowledge about "how the world works" is called a model of the world.
- An agent that uses such a model is called a model-based agent.
- Internal state - percept history
- Requires two knowledge,
  - First, some information about how the world evolves independently of the agent
  - Second, some information about how the agent's own actions affect the world

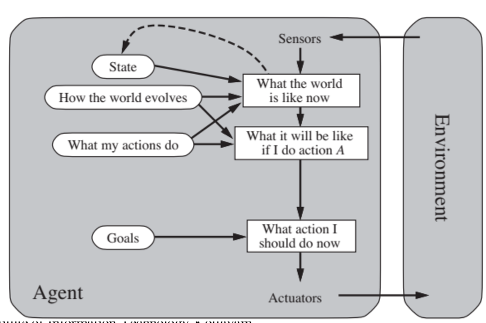


A self-parking car uses sensors plus an internal model of the car's position to park correctly.

Goal-based agents

A navigation system (like Google Maps) that selects routes to reach a destination

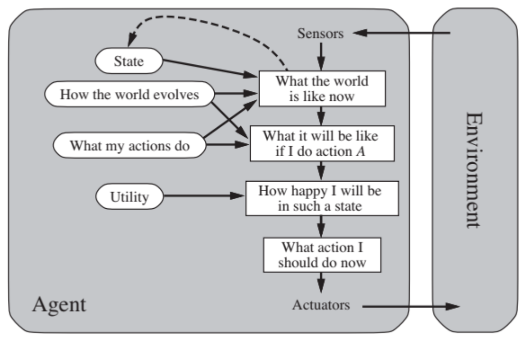
- Knowing something about the current state of the environment is not always enough to decide what to do.
  - at a road junction, the taxi can turn left, turn right, or go straight on.
- the agent needs some sort of goal information that describes situations that are desirable
  - being at the passenger's destination
- Goal-based action selection,
  - straightforward
  - tricky
- Search - subfields of AI devoted to finding action sequences that achieve the agent's goals.



Utility-based agents

- Utility - general performance measure should allow a comparison of different world states according to exactly how happy they would make the agent.
- Maximize the utility function that measures its preferences among states of the world.
- it chooses the action that leads to the best expected utility
- An agent's utility function is essentially an internalization of the performance measure

An e-commerce recommendation system that suggests products to maximise customer satisfaction and profit



Learning agents

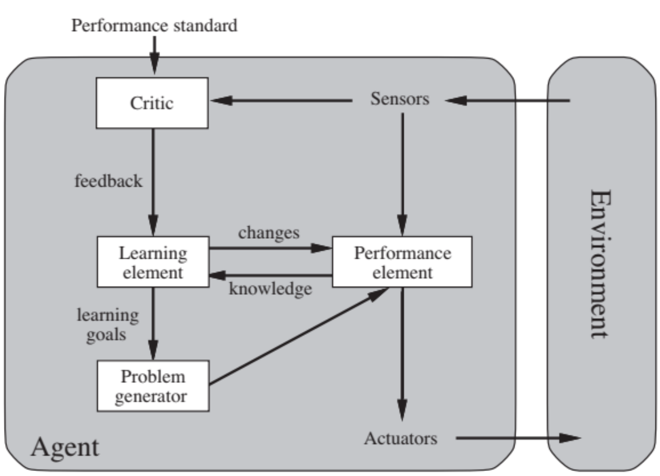
- learning element is responsible for making improvements
- performance element is responsible for selecting external actions.
- The learning element uses feedback from the critic on how the agent is doing
  - determines how the performance element should be modified to do better in the future
- critic tells the learning element how well the agent is doing with respect to a fixed performance standard
- problem generator suggesting actions that will lead to new and informative experiences
  - job is to suggest these exploratory actions

Learning agents

- ability to learn from experiences and improve performance
- *Learning Element*: Modifies the agent's behavior based on past experiences.
- *Performance Element*: Chooses actions based on percepts and learned knowledge.
- *Critic*: Provides feedback on the agent's performance.
- *Problem Generator*: Suggests actions that might lead to new and informative experiences.
  - The learning element uses feedback from the critic on how the agent is doing to a fixed performance standard
  - determines how the performance element should be modified to do better in the future

AlphaGo, which learned to play Go better than humans using reinforcement learning

Learning agents



Agent Programs

Agent Type	Definition	Working Principle	Example	Key Features
Simple Reflex Agent	Acts only on the current percept using condition–action rules.	If condition matches percept → take corresponding action.	Thermostat: turns heater on/off depending on temperature.	Fast, simple, works well in fully observable environments.
Model-Based Reflex Agent	Maintains an internal model of the world to handle partial observability.	Updates internal state from percepts and history → decides action.	Self-parking car: uses sensor data + memory of past movements.	Handles incomplete info, tracks world state.
Goal-Based Agent	Chooses actions to achieve specific goals.	Considers future outcomes → selects action sequence to reach goal.	Google Maps: plans shortest/fastest route to destination.	Flexible, goal-oriented, adaptive to changing goals.
Utility-Based Agent	Evaluates multiple alternatives and chooses the one with the highest utility.	Uses a utility function to compare states and outcomes.	E-commerce recommender: suggests products maximizing satisfaction/profit.	Optimizes decisions, balances trade-offs.
Learning Agent	Improves performance through experience and feedback.	Uses feedback (reward/punishment) to adapt and learn over time.	AlphaGo: learned to play Go and defeated human champions.	Adaptive, can handle new/unseen situations, improves over time.

State Representation

- Three ways to represent states and the transitions between them,
- Atomic: a state is a black box with no internal structure
  - The entire state is treated as a single, indivisible black box
  - each state of the world is indivisible—it has no internal structure
  - search and game-playing - work with atomic representations
  - Usage: Common in search algorithms (like BFS, DFS, A*) and game-playing agents (like Chess/Checkers AI).
  - Example: A chess board position is considered as one atomic state without explicitly describing piece-by-piece details.

State Representation

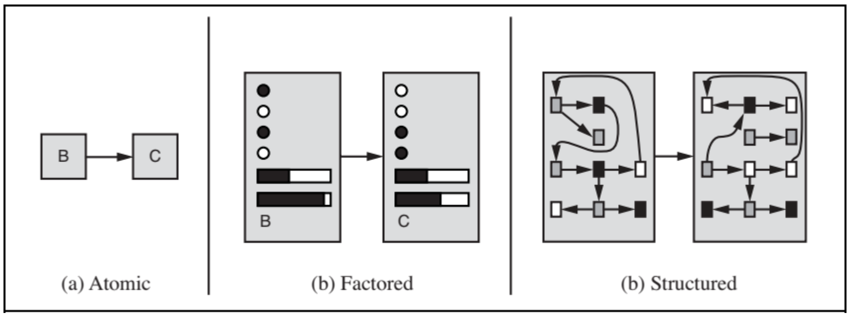
- Factored: a state consists of a vector of attribute values
  - The state is described as a vector of attributes or variables, each with values.
  - splits up each state into a fixed set of variables or attributes
  - values can be Boolean, real valued, or one of a fixed set of symbols
  - widely used in propositional logic, machine learning models, and planning algorithms
  - Example: Weather state = {Temperature = 30°C, Humidity = 80%, Raining = Yes}.
    - Each factor is stored separately.

State Representation

- Structured: a state includes objects, each of which may have attributes of its own as well as relationships to other objects.
  - The state is represented in terms of objects, attributes, and relationships.
  - Captures more detail, allows reasoning over entities and their interactions.
  - Common in relational databases, knowledge graphs, and first-order logic reasoning
  - Example: In a hospital database: *Doctor(Alice) treats Patient(Bob)*

Here “Alice” and “Bob” are objects, with relationships defined between them.

State Representation



State Representation

Representation Type	Description	Example	Pros	Cons
Atomic Representation	Treats each state as a unique, indivisible entity.	Chessboard configurations, Maze positions	Simple, easy to implement	Not flexible or scalable for complex states
Factored Representation	Represents states using a set of variables/ attributes.	Robotics, Strategy games, Variables like time slots, rooms, and tasks in scheduling.	Compact, flexible, allows detailed analysis	More complex to implement
Structured Representation	Represents states as a collection of objects and their relationships.	A hierarchy of objects and relationships in robotics, AI planning	Highly expressive, supports complex reasoning	Computation ally expensive, difficult to manage

The End…